



IT6105 - SOFTWARE DEVELOPMENT PROJECT GUIDELINES

2025



**Degree of Bachelor of Information Technology of the
University of Colombo School of Computing**



Table of Contents

1. Introduction	3
1.1. Intended Learning Outcomes (ILOs)	3
2. Project Phases	4
2.1. Project Proposal	4
2.2. Interim submission	6
2.3. Final submission, presentation, and demonstration	7
3. Project Types	7
3.1. Web Application	7
3.2. Mobile Application	7
3.3. Desktop Application	8
3.4. Cloud-based Systems (NOT ALLOWED)	8
3.5. Data-Driven Systems, AI/Machine Learning-based Projects (NOT ALLOWED)	8
3.6. Other Types (e.g., Games, IoT, Security Applications)	8
4. Supervisor and Responsibilities	9
5. Project Requirements	9
5.1. Functional Requirements	10
5.2. Non-Functional Requirements (Usability, Performance)	10
5.3. Constraints and Limitations	10
5.4. Tools and Technologies	11
6. System Design	11
6.1. High-Level System Architecture	12
6.2. Use Case Diagrams and Specifications	12
6.3. Class Diagrams, Sequence Diagrams	13
6.3.1. State Transition Diagrams (State Diagrams)	13
6.4. Alternative Diagrams (Non-OOP based development)	14

6.4.1. Data Flow Diagrams (DFDs)	14
6.4.2. Flowcharts	14
6.4.3. Structured Charts	14
6.4.4. Functional Decomposition Diagrams	15
6.4.5. Control Flow Diagrams	15
6.5. Database Design and ER Diagrams	15
6.5.1. Entity-Relationship Diagrams (ERDs)	15
6.6. User Interface Design (Wireframes, Mockups)	16
7. Implementation and Development	16
7.1. Coding Standards and Guidelines	16
7.2. Version Control (e.g., Git)	17
7.3. Recommended Software Development Methodologies (Agile, Waterfall)	17
8. Testing and Quality Assurance	18
8.1. Unit Testing	19
8.2. Integration Testing	19
8.3. User Acceptance Testing (UAT)	20
8.4. Quality Metrics	20

1. Introduction

The Final Year Project of the BIT degree program is designed to help students gain practical experience in software development by following a structured approach aligned with industry best practices. The project carries **8 credits** and requires approximately **300 hours of work**. Students may choose to develop a system for a real client, although this is not mandatory, and they also have the flexibility to propose their own project topics. In either case, the selected topic and developed system must meet the requirements set for the Final Year Project of the BIT degree program. Each student must also find a qualified supervisor with a recognized degree in Information Technology or Computer Science from a **UGC-accredited institution**.

1.1. Intended Learning Outcomes (ILOs)

Upon successful completion of the Final Year Project, the student will be able to:

1. **Apply** software development best practices to plan, design, implement, test, and deploy a software system.
2. **Prepare** comprehensive project documentation covering all stages of the software development lifecycle.
3. **Demonstrate** the ability to independently manage a complete software project within defined time and resource constraints.
4. **Present** project outcomes effectively through written reports, visual aids, and oral presentations.
5. **Evaluate** the quality, usability, and performance of the developed system using appropriate testing and validation techniques.
6. **Reflect** on individual learning, challenges, and professional growth gained through the project experience.

2. Project Phases

2.1. Project Proposal

Before starting the project work, the candidate must register for the project and submit a project proposal through the VLE. The project proposal outlines the planned work to be carried out during the project period and serves as the foundation for defining the project scope, ensuring it represents approximately 300 hours of individual effort.

The candidate should discuss the proposed project with the selected supervisor before submitting the proposal. A copy of the submitted proposal should be kept for future reference. The completed proposal must be uploaded to the VLE by the deadline specified in the published schedule.

Please note that the Project Evaluation Board (PEB) will not provide feedback on proposed projects. Candidates must also complete the Supervisor Agreement Form (available on the VLE), have it signed by their supervisor(s), and upload a scanned PDF copy to the VLE before the given deadline.

The proposal should include clearly defined system features. If any existing frameworks, libraries, or templates are used, the work already implemented in such frameworks will **not** be counted toward the student's individual contribution. The project must reflect a substantial amount of original design, development, and testing effort by the candidate.

Sample structure for the proposal is provided below.

Project Title

- A concise and descriptive title reflecting the problem or solution domain of the individual software development project.

Student and Supervisor Details

- Student name, registration number, email

- Supervisor name, qualification, designation, and contact details

Background and Problem Description

- Description of the context or problem being addressed.
- Why this problem or system is important or relevant.

Objectives

- Specific goals the project aims to achieve through the software application developed.

Proposed System and Key Features

- Overview of the proposed solution or system architecture (use a high level component diagram).
- **List of clearly defined system features**, specifying the main functionalities to be implemented by the student. Each feature should clearly list the functionalities that are developed by the student to complete the feature. An example for a single feature is provided below.
 - **Feature : Book Catalog Management**
 - Functionalities developed by the student:
 - Add, edit, and delete book records (title, author, price, category, stock quantity)
 - Search and filter books by title, author, category, or price range
 - Display detailed book information with images and descriptions
 - Upload and manage book cover images
 - Implement pagination for large book lists
 - Track book availability (in stock / out of stock)
 - Sort books by name, price, or popularity
 - Allow admin to bulk import/export book data in CSV format
 - Validate input data to prevent incomplete or invalid entries

- Implement audit logs for book additions or updates (*Plan to use a library for log file generation*)
- Indicate any third-party components, frameworks, or libraries used and clearly distinguish the student's original contribution by mentioning them alongside with the respective function.

Tools and Technologies

- Programming languages, development frameworks, databases, and tools to be used and justifications for using them.

Estimated Work Plan (300 hours)

- A brief timeline or task allocation should be provided, covering the phases of planning, design, implementation, testing, documentation, and presentation. The work plan must clearly indicate the estimated effort (in hours) for each phase of the software development process. Under the implementation phase, the estimated effort should be broken down by each proposed feature to show how the total workload is distributed across the project.

Expected Deliverables

- Fully functional software system, interim submissions, final dissertation, presentation materials.

2.2. Interim submission

The interim report aims to ensure that you understand the project objectives, have mastered the client requirements, have done a good design of your solution and have a realistic time plan to complete the project on time. A separate template and guideline document is provided for further details.

2.3. Final submission, presentation, and demonstration

At the end of the project, the candidate must submit all required deliverables through the VLE by the specified deadline. The final submission should include the complete software system, related documentation, and any other materials requested in the project guidelines.

Each candidate must also make a formal **presentation and demonstration** of their project to the evaluation panel. The presentation should summarize the project objectives, design, implementation, and key features, while the demonstration should show the system's functionalities in operation. The student should be prepared to answer questions regarding design decisions, implementation details, and testing outcomes to demonstrate individual understanding and contribution.

3. Project Types

When selecting a final year project, students have the flexibility to choose from a variety of project types, depending on their interests and technical expertise. Each project type offers unique challenges and opportunities for students to demonstrate their skills in software development and problem-solving. Below are the common types of projects students can undertake:

3.1. Web Application

A Web Application is software that runs on a web server and is accessed through a web browser. These applications are often interactive and can serve various purposes, such as e-commerce, content management, online services, etc.. Web applications typically involve both front-end (UI/UX design) and back-end (server-side logic) development, making them suitable for students who want to demonstrate full-stack development skills. Common technologies include HTML, CSS, JavaScript, PHP, Python, and various frameworks like React, Angular, Laravel, etc..

3.2. Mobile Application

A Mobile Application is designed to run on smartphones or tablets and can be developed for platforms like Android or iOS. These projects can focus on building native apps using languages like Swift (for iOS) or Kotlin/Java (for Android), or cross-platform apps using frameworks like Flutter or React Native. Mobile applications provide an opportunity to explore user interface

design, mobile-specific functionality (e.g., GPS, push notifications), and app store deployment processes.

3.3. Desktop Application

A Desktop Application is software that is installed and runs on a desktop or laptop computer. These applications are often used for specialized tasks like graphic design, accounting, or data analysis. Desktop application development requires knowledge of programming languages such as Java, C#, or Python, and often involves building complex user interfaces and handling file or system operations. This type of project is well-suited for students who prefer working in a desktop environment over web or mobile platforms.

3.4. Cloud-based Systems (NOT ALLOWED)

Cloud-based Systems leverage cloud infrastructure for data storage, processing, and scalability. These projects can involve building applications that use cloud platforms like AWS, Microsoft Azure, or Google Cloud to deliver software-as-a-service (SaaS), platform-as-a-service (PaaS), infrastructure-as-a-service (IaaS), etc.. The applications using Cloud-based technologies are not accepted as those applications can be developed in such a way that they are not well aligned with the program ILOs. Further, evaluation restrictions and limitations are also considered when excluding Cloud-based systems / projects.

3.5. Data-Driven Systems, AI/Machine Learning-based Projects (NOT ALLOWED)

AI/Machine Learning-based Projects involve creating systems that use artificial intelligence or machine learning to solve complex problems. These projects typically require knowledge of algorithms, data processing, and machine learning frameworks. The applications or software that focus on such data driven approaches, AI/ML projects are not accepted as the final year project of the program as those solutions or approaches are not well aligned with the program ILOs.

3.6. Other Types (e.g., Games, IoT, Security Applications)

In addition to traditional software applications, students may also choose to work on specialized project types such as:

- **Games:** Developing interactive video games using game engines like Unity or Unreal Engine. These projects demonstrate creativity, physics modeling, and user engagement.
- **Internet of Things (IoT):** Building systems that integrate physical devices with software to create smart systems, such as home automation, wearable devices, or industrial sensors.
- **Security Applications:** Focusing on cybersecurity challenges, such as developing encryption algorithms, intrusion detection systems, or secure communication protocols.

However, working on this type of projects would pose a risk as if the project work does not have sufficient software development contribution, you may get failed at the final project evaluation.

4. Supervisor and Responsibilities

Students shall find a suitable supervisor based on the supervisor selection criteria published on the Project VLE. Once the project proposal is approved, the supervisor will guide the student throughout the project. The supervisor's role includes:

- Providing feedback on the proposal and project direction.
- Meeting regularly with the student to monitor progress.
- Offering advice on technical issues, project management, and documentation.
- Ensuring that the student adheres to the project scope and timeline.
- Reviewing the final dissertation and verifying that the project meets academic and professional standards required by BIT program.

Students should maintain open communication with their supervisor, seek guidance as needed, and incorporate their feedback to ensure project success.

5. Project Requirements

For a successful final year project, students must carefully define and meet the project requirements. These requirements are divided into functional and non-functional categories, alongside constraints, limitations, and the tools or technologies chosen for the project. Clearly understanding and documenting these aspects ensures that the project delivers value to both users and clients while meeting academic standards.

5.1. Functional Requirements

Functional Requirements define the core functionalities that the system must provide. These are specific actions or tasks that the system will perform to meet user or business needs. Examples include:

- User registration and authentication
- Data input, processing, and storage (e.g., entering product details, generating reports)
- CRUD (Create, Read, Update, Delete) operations for managing data
- Interaction with external systems or APIs (e.g., payment gateways, third-party services)
- Business logic to support workflows (e.g., order management, content approval processes)

Functional requirements should be clear, measurable, and testable, ensuring that they can be validated through system testing and user acceptance testing. Note that security is considered as a functional requirement.

5.2. Non-Functional Requirements (Usability, Performance)

Non-Functional Requirements describe the quality attributes of the system and ensure that it operates efficiently, and is user-friendly. These typically include:

- **Usability:** Ensures that the system is intuitive and easy to use for its target audience. This involves a clean user interface, clear navigation, and responsiveness.
- **Performance:** Defines how the system behaves under certain conditions, including speed of operation, response times, and ability to handle concurrent users or large datasets.

These requirements enhance user satisfaction and protect the system from vulnerabilities.

5.3. Constraints and Limitations

Constraints and Limitations refer to any restrictions or boundaries that impact the development or performance of the system. These could be technical, environmental, or resource-based, such as:

- **Budget Constraints:** Limits on financial resources that may restrict the use of certain tools or platforms.
- **Time Constraints:** Deadlines for project completion that affect the scope of work and feature development.

- **Technical Constraints:** Compatibility with legacy systems or hardware, or limitations in bandwidth, storage, or processing power.
- **Legal and Regulatory Limitations:** Requirements to comply with laws and regulations (e.g., GDPR for data privacy, accessibility standards).

Identifying these constraints early helps manage expectations and ensures that realistic goals are set.

5.4. Tools and Technologies

The **Tools and Technologies** used in the project play a significant role in determining the project's success. Selecting the right stack depends on the project's nature, the student's expertise, and the requirements of the system. Typical tools and technologies include:

- **Programming Languages:** Java, Python, C#, PHP, JavaScript, etc., depending on the type of application (web, mobile, desktop).
- **Frameworks and Libraries:** React, Angular, Spring, Laravel, Flutter, or Django for rapid development and best practices.
- **Database Systems:** MySQL, PostgreSQL, MongoDB, or Firebase for data storage and management.
- **Development Tools:** Version control systems like Git, IDEs (e.g., VSCode, IntelliJ), and collaboration platforms like GitHub.
- **Testing Tools:** Selenium, JUnit, or TestNG for automated testing, and JIRA for project management.
- **Cloud Services:** AWS, Azure, or Google Cloud for hosting, scalability, and cloud-based services. **(NOT ALLOWED)**

6. System Design

System design is a critical phase in software development that translates the project's requirements into a comprehensive blueprint for implementation. This phase ensures that the system is well-structured, functional, and capable of meeting user needs. The system design phase covers both high-level architectural decisions and detailed design elements, enabling the development of a scalable, maintainable, and efficient solution.

6.1. High-Level System Architecture

The High-Level System Architecture provides an overview of the entire system, describing its main components, how they interact, and how the system integrates with external systems or services. This design typically includes:

- **Component and Module Breakdown:** Describes the different modules within the system (e.g., authentication, user management, reporting).
- **Integration Points:** Specifies how external systems (e.g., APIs, third-party services) are integrated into the system.
- **Component interaction:** Specifies how internal components communicate and interact with each other.
- **Technology Stack:** Identifies the main technologies, frameworks, and tools used in the system (e.g., Angular for front-end, Node.js or Java for back-end, MySQL for the database).

This high-level view helps to ensure the system is designed with scalability, flexibility, and future growth in mind. Example architecture styles include layered architecture, client-server architecture, microservices architecture, event-driven or message based architecture, etc..

6.2. Use Case Diagrams and Specifications

Use Case Diagrams are used to visually represent the system's functionality from a user's perspective. Each use case represents a specific interaction between the user (or another system) and the system itself to achieve a particular goal. Use case diagrams help in:

- Identifying key **actors** (e.g., users, administrators, external systems) involved in the system.
- Highlighting the **use cases** (tasks) performed by each actor, such as "Login," "Place Order," or "Generate Report."
- Defining **specifications** for each use case, which describe the flow of actions, inputs, and expected outcomes for each use case.

Use case diagrams ensure that the system meets user needs and helps developers focus on delivering the necessary functionality.

6.3. Class Diagrams, Sequence Diagrams

Class Diagrams and Sequence Diagrams provide more detailed insights into the system's design and behavior.

- **Class Diagrams:** These diagrams outline the system's structure by showing classes (objects), their attributes, methods, and the relationships between them (e.g., inheritance, association). Class diagrams define how data is modeled and how different parts of the system interact.
 - Example: In an e-commerce system, classes might include User, Product, Order, and Payment. The relationships between these classes define how they work together to achieve system functionality.
- **Sequence Diagrams:** These diagrams illustrate the flow of messages or interactions between objects or components over time. They are used to model the dynamic behavior of the system.
 - Example: In the same e-commerce system, a sequence diagram might show how a user browses products, adds items to a cart, and completes a purchase. It visually maps the interaction between the User, Shopping Cart, Payment, and Order components.

Together, class and sequence diagrams help developers understand both the static and dynamic aspects of the system.

6.3.1. State Transition Diagrams (State Diagrams)

- **Purpose:** To describe the states an entity can be in and how it transitions between those states based on events or conditions.
- **Components:**
 - **States:** Represent various conditions an entity can be in (e.g., Idle, Processing, Completed).
 - **Transitions:** Arrows that show how the system moves from one state to another due to events or actions.

Example: A state diagram for an online ordering system could include states like "Order Placed," "Processing," "Shipped," and "Delivered."

6.4. Alternative Diagrams (Non-OOP based development)

If the system development is not object-oriented, the design diagrams would focus on structural and functional aspects rather than objects, classes, and their interactions. In this case, the following diagrams are commonly used for structured or procedural system design:

6.4.1. Data Flow Diagrams (DFDs)

- **Purpose:** To illustrate how data flows through the system, including inputs, processing, and outputs.
- **Levels:**
 - **Context Diagram (Level 0 DFD):** A high-level view showing the system and its interactions with external entities (users, databases, etc.).
 - **Detailed DFD (Level 1, Level 2, etc.):** Breaks down processes into more detailed components, showing specific data flows within the system.

Example: A DFD for a library management system would show how data flows between the user, the book catalog, and the borrowing/returning process.

6.4.2. Flowcharts

- **Purpose:** To visually represent the step-by-step sequence of operations in a process or algorithm.
- **Elements:**
 - **Processes:** Represented by rectangles, showing tasks or actions.
 - **Decisions:** Represented by diamonds, indicating branching logic based on conditions.
 - **Inputs/Outputs:** Represented by parallelograms, showing where data enters or leaves the system.

Example: A flowchart for a login process would outline steps such as entering credentials, checking validity, and granting or denying access.

6.4.3. Structured Charts

Purpose: To depict the hierarchical breakdown of system modules, showing how different functions interact.

- **Components:**
 - **Modules:** Represent individual tasks or functions, arranged in a hierarchy.
 - **Lines:** Indicate the flow of control between modules.

Example: A structured chart for a payroll system would break down the main payroll process into sub-processes like "Calculate Salary," "Deduct Taxes," and "Generate Payslip."

6.4.4. Functional Decomposition Diagrams

- **Purpose:** To break down a complex system or process into simpler, more manageable components.
- **Elements:**
 - **Main Function:** The top-level function or process.
 - **Sub-functions:** The decomposition of the main function into smaller, more specific functions.

Example: A functional decomposition diagram for an ATM system would show high-level functions like "Withdraw Cash" and "Check Balance," with further breakdown into steps like "Insert Card" and "Enter PIN."

6.4.5. Control Flow Diagrams

- **Purpose:** To show the order of execution of processes and how control flows through a system.
- **Elements:** Processes, decisions, and paths showing the execution sequence.

Example: A control flow diagram for a traffic light system would show the sequence in which lights change based on control logic and timing.

6.5. Database Design and ER Diagrams

6.5.1. Entity-Relationship Diagrams (ERDs)

- **Purpose:** To model the data structure of the system, showing relationships between different data entities.
- **Components:**
 - **Entities:** Represent real-world objects or concepts (e.g., Customers, Products).
 - **Attributes:** Properties of entities (e.g., Customer Name, Product Price).

- Relationships: Show how entities are related (e.g., Customers place Orders).

Example: An ERD for an e-commerce platform would depict entities like Users, Products, Orders, and their interrelations. Only use Chen Notation for the ERD.

6.6. User Interface Design (Wireframes, Mockups)

User Interface (UI) Design focuses on creating an intuitive and aesthetically pleasing interface for end users. This step involves creating:

- **Wireframes:** Low-fidelity, skeletal blueprints that show the layout and structure of the system's user interface. Wireframes highlight the placement of buttons, menus, forms, and other interactive elements without focusing on design details.
 - Example: A wireframe for a login page might show the position of the username and password fields, login button, and forgot password link.
- **Mockups:** High-fidelity, visually detailed representations of the user interface that include design elements like color schemes, fonts, and branding.
 - Example: A mockup of the e-commerce homepage could include the final product images, banners, color schemes, and font styles to give a clear view of the intended look and feel.

Wireframes and mockups help ensure that the final system will be user-friendly and visually appealing while meeting usability standards.

7. Implementation and Development

The Implementation and Development phase is where the system design is translated into actual code, bringing the project to life. This phase involves following coding standards, using version control systems, adhering to software development methodologies, and ensuring effective collaboration if working in a team. Below are key aspects to ensure successful implementation.

7.1. Coding Standards and Guidelines

Following **Coding Standards** ensures that the code is clean, consistent, and maintainable. Adhering to best practices for writing code not only improves readability but also helps in future updates and debugging. Important coding guidelines include:

- **Consistent Naming Conventions:** Use meaningful names for variables, functions, and classes. Follow established naming patterns (e.g., camelCase, snake_case) depending on the language and framework.
- **Code Readability:** Write code with proper indentation, spacing, and comments. Clear and concise comments explain the purpose of functions and critical sections of code.
- **Modularization:** Break the code into smaller, reusable modules or functions to avoid repetition and enhance maintainability.
- **Error Handling:** Implement robust error handling to manage exceptions, unexpected inputs, and failures gracefully.
- **Security Considerations:** Follow security best practices, such as input validation, avoiding SQL injection, and securing sensitive data.

Adhering to coding standards ensures that the project is scalable and that future developers (or the student themselves) can easily understand and extend the system.

7.2. Version Control (e.g., Git)

Version Control is essential for tracking changes to the codebase, collaborating with others, and maintaining a history of the project. **Git**, a popular version control system, provides:

- **Branching:** Allows developers to work on new features or bug fixes in isolated branches without affecting the main codebase.
- **Commit History:** Tracks all changes made to the project, making it easy to revert to previous versions if needed.
- **Collaboration:** Facilitates teamwork by allowing multiple contributors to work on different parts of the project simultaneously, and merging their contributions without conflict.
- **Backup:** Ensures that code is safely stored in a remote repository (e.g., GitHub, GitLab) in case of local machine failure.

Using Git or another version control system is a fundamental practice in modern software development that enhances productivity, accountability, and teamwork.

7.3. Recommended Software Development Methodologies (Agile, Waterfall)

Selecting the right Software Development Methodology is crucial to ensure that the project progresses smoothly from conception to completion. Two common methodologies are:

- **Agile Methodology:** Agile is an iterative, flexible approach where development occurs in short cycles called "sprints". Agile is mostly used when the requirements are dynamic or not very clear. Key features include:
 - **Regular feedback:** Frequent client or user feedback at the end of each sprint helps refine the system continuously.
 - **Adaptability:** Agile allows for changes and new requirements to be incorporated throughout the development process.
 - **Collaboration:** Encourages close collaboration between developers, testers, and stakeholders, ensuring that the project meets evolving needs.

Agile is ideal for projects with unclear or changing requirements, where regular input is needed.

- **Waterfall Methodology:** Waterfall is a more traditional, linear approach where each phase of the project (requirements, design, development, testing) is completed before moving to the next phase. Waterfall methodology is used when the requirements are fixed and clear. Key features include:
 - **Defined stages:** Each phase has clearly defined tasks, and once a phase is completed, it is difficult to revisit it.
 - **Predictability:** Since the requirements and design are finalized upfront, the project follows a predictable path.

Waterfall is suited for projects with well-defined, unchanging requirements where the scope is unlikely to evolve during development.

Students can choose the methodology that best fits the nature of their project and their working style, though Agile is typically preferred in dynamic, real-world environments.

8. Testing and Quality Assurance

The **Testing and Quality Assurance (QA)** phase ensures that the system functions as intended, meets user requirements, and maintains high standards of performance, security, and reliability. This phase involves various testing strategies to identify and fix bugs, optimize the system, and validate its effectiveness. Below are the key components of this phase.

8.1. Unit Testing

Unit Testing involves testing individual components or modules of the software to ensure that each part functions correctly in isolation. Typically, developers write unit tests to validate the behavior of specific functions or methods.

- **Purpose:** Detect bugs early in development, ensuring that individual units of the code work as expected.
- **Automation:** Unit tests are often automated, using testing frameworks like **JUnit** (for Java), **PyTest** (for Python), or **PHPUnit** (for PHP), making them easy to run frequently during development.
- **Granularity:** Unit tests are highly granular, focusing on small sections of code. For example, testing a function that calculates the total cost of items in a shopping cart.

Regularly conducting unit tests ensures that the system is built on solid, bug-free foundations, reducing the risk of major failures later in the development process.

8.2. Integration Testing

Integration Testing focuses on testing how different components or modules of the system work together. While unit testing verifies individual parts, integration testing ensures that these parts interact correctly when combined.

- **Purpose:** Identify issues with the interaction between different modules (e.g., front-end communicating with back-end, or database queries fetching the correct data).
- **Scope:** Covers the interfaces between modules, ensuring data flows correctly and modules communicate without errors.
- **Example:** In an e-commerce system, testing how the product search functionality interacts with the inventory database, ensuring that search results display accurate stock levels.

By running integration tests, developers can catch issues that arise when integrating different system components, ensuring smooth and correct interactions across the system.

8.3. User Acceptance Testing (UAT)

User Acceptance Testing (UAT) is performed by the end users to verify that the system meets their needs and is ready for deployment. UAT is the final phase of testing and focuses on validating the system's functionality against real-world use cases.

- **Purpose:** Ensure that the system meets the business requirements and is usable in real-world scenarios.
- **Process:** End users or clients interact with the system, providing feedback on whether the system performs as expected. UAT tests may involve real-world tasks like placing an order or generating reports.
- **Sign-off:** A successful UAT typically results in formal sign-off by the client or stakeholders, indicating that the system is ready for release.

8.4. Quality Metrics

Quality Metrics provide objective measurements of the system's quality, helping to assess how well the system performs in terms of functionality, usability, performance, and reliability. Common quality metrics include:

- **Code Coverage:** Measures the percentage of code that is covered by unit and integration tests. A higher percentage indicates more comprehensive testing.
- **Defect Density:** The number of defects found per 1,000 lines of code. A lower defect density indicates higher code quality.
- **Mean Time to Failure (MTTF):** The average time between failures of the system, indicating reliability.
- **Mean Time to Repair (MTTR):** The average time taken to fix bugs or issues. A lower MTTR suggests faster bug resolution.
- **User Satisfaction:** Feedback from UAT or post-deployment surveys to gauge how satisfied users are with the system.